Computer Programming – (Unit – 4)

Syllabus

Structures – Arrays and Structures – nested structures – passing structures to functions – user defined data types – Union. Pointers – pointers and arrays – pointers and functions - pointers and strings - pointers and Structures.

PART –A

2 marks

1. **Explain parameter passing by reference.(May 2014) (May 2017)**

   Pass-by-reference means to pass the reference of an argument in the calling function to the corresponding formal parameter of the called function. The called function can modify the value of the argument by using its reference passed in. The following example shows how arguments are passed by reference.

2. **Give an example where a structure data type may be required. (May 2014)**

   Example:

```
struct lib_books
{
char book_name[50];
int pages;
float price;
};
```

   The book_name, number of pages and price are of distinct data types and hence structure is used in this example.

3. **Compare and contrast between structure and union.(Jan 2015) (May 2016) (Dec 2017)**

Structure	**Union**
Structure is the container defined in C to store data variables of different type and also supports for the user defined variables storage.	On other hand Union is also similar kind of container in C which can also holds the different type of variables along with the user defined variables.
Structure in C is internally implemented as that there is separate memory location is allotted to each input member	While in case Union memory is allocated only to one member having largest size among all other input variables and the same location is being get shared among all of these.

Syntax: struct struct_name{ type element1; type element2; . . } variable1, variable2, ...;	Syntax: union u_name{ type element1; type element2; . . } variable1, variable2, ...;

4. What is a self – referential structure? Give Example. (Jan 2016)

Self-Referential structures are those structures that have one or more pointers which point to the same type of structure, as their member. In other words, structures pointing to the same type of structures are self-referential in nature.

5. What is a pointer? List out the advantages of using pointer. (Jan 2016) (Dec 2017) (May 2018) (May 2019)

A pointer is a variable that holds a memory address. This address is the location of another object (typically another variable) in memory. For example, if one variable contains the address of another variable, the first variable is said to point to the second.

→Generally computer uses memory for storing instruction and values of the variables within the program, but the pointers has memory address as their values.
→ The memory address is the location where program instructions and data are stored, pointers can be used to access and manipulate data stored in the memory.

6. How strings are represented in C language? (May 2016)

Strings in C are represented by arrays of characters. The difference between a character array and a string is the string is terminated with a special character '\0'.

Declaring a string is as simple as declaring a one-dimensional array.

Syntax:
char str_name[size];

7. Differentiate between user defined types and standard data types. (Dec 2016)

Standard data types	**User defined data types**
Standard data types are already defined inside the language i.e , user can use these	User defined data types are the data types which user have to define while or before

data types without defining them.	using them.
	A user-defined data type (UDT) is a data type that derived from an existing data type. You can use UDTs to extend the built-in types already available and create your own customized data types.
Example: integer , character , void , float etc.	Example: In C language , structure and union are used as user-defined data types.

8. Give the applicability of union in programming.(May 2017)

Unions are mostly used in embedded programming where direct access to the memory is needed. Unions can be useful in many situations where we want to use the same memory for two or more members.

9. Define union. (May 2018) (May 2019) (Dec 2019)

- Union is a derived data type and it is declared like structure.
- The difference between union and structure is in terms of storage.
- In structure each member has its own storage location, whereas all the members of union use the same location, although a union may contain many members of different types.
- When we use union the compiler allocates a piece of storage that is large enough to hold.
- Union is also declared by using the keyword union.

10. What are structures? State how structures members are accessed? (Dec 2018)

- A structure is an organized collection of different types of elements. A structure is also called as a record. Elements of a structure are called the members or fields. These fields may be of int, float, char or double.
- The struct is a keyword to provide a structure facility in C.
- Members in a structure are accessed using the Period Operator '.'.

11. Write a program to calculate the sum of digits using pointers. (Dec 2018)

```
#include <stdio.h>
int main()
{
  int first, second, *p, *q, sum;
  printf("Enter two integers to add\n");
```

```
 scanf("%d%d", &first, &second);
  p = &first;
  q = &second;
  sum = *p + *q;
  printf("Sum of the numbers = %d\n", sum);
  return 0;
}
```

12. What is meant by user defined data types? Give examples. (Dec 2019) (Sep 2020)

C provides, typedef (for type definition) statement that enables the programmer to define an alternate term to the basic data types. Once the typedef is made, the earlier datatype, all variables, arrays, structures etc., can be declared with the new data type.

Syntax:

typedef old_data_type new_data_type;

typedef - is a keyword

old_data_type - a basic data type

new_data_type - user-defined data type

13. What is meant by structure within structure? (Sep 2020)

Structure within another structure is nothing but nesting of structures (i.e., a structure can contain one or more structures as its members). A structure may be defined and/or declared inside another structure.

Example:

```
struct stud_Res
{
 int rno;
char nm[50];
char std[10];
struct stud_subj
{
char subjnm[30];
int marks;
}subj;
}result;
```

PART-B (11 marks)

1. Write a C program to find the maximum of an array of integer numbers using pointer. (May 2014)

Program:

```
#include <stdio.h>
int main()
{
        long array[100], *maximum, size, c, location = 1;
        printf("Enter the number of elements in array\n");
        scanf("%ld", &size);
        printf("Enter %ld integers\n", size);
        for ( c = 0 ; c < size ; c++ )
        {
            scanf("%ld", &array[c]);
        }
        maximum  = array;
        *maximum = *array;
        for (c = 1; c < size; c++)
        {
             if (*(array+c) > *maximum)
           {
             *maximum = *(array+c);
             location = c+1;
           }
        }
      printf("Maximum element is present at location number %ld and it's value is %ld.\n",
      location, *maximum);
      return 0;
   }
```

2. a. Discuss nested structures with an example.(May 2014)

- Nesting of structures are nothing but **structure within another structure** (i.e., a structure can contain one or more structures as its members).
- A structure may be defined and/or declared inside another structure.

Syntax	**Example**
struct structure_name1 { <data-type> member 1;	struct stud_Res { int rno;

```
        <data-type> member 2;
        - - - - - - - - - - -
        - - - - - - - - - - -
        <data-type> member n;
        struct structur_ name2
        {
                <data-type> member 1;
                <data-type> member 2;
                - - - - - - - - - - -
                - - - - - - - - - - -
                <data-type> member n;
        }inner_struct_var;
}outer_struct_var;
```

```
        char nm[50];
        char std[10];

        struct stud_subj
        {
                char subjnm[30];
                int marks;
        }subj;
}result;
```

Description:

- In above example, the structure stud_Res consists of stud_subj which itself is a structure with two members.
- Structure stud_Res is called as '**outer structure**' while stud_subj is called as '**inner structure.**'
- The members which are inside the inner structure can be accessed as follow :

  result.subj.subjnm

  result.subj.marks

Example program:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    struct stud
    {
        int rollno;
        char sname[30];
        struct dob
        {
            int date,mon,year;
         }d;
        int m1,m2,m3, tot;
        float avg;
        }s[10];
int n, i;
printf("Enter the number of students");
```

```
scanf(%d",&n);
printf("\enter the student details\n");
for(i=1;i<=n;i++)
{
printf("\nenter the id and value:");
scanf("%d%s",&s[i].rollno,s[i].sname);
printf("\nenter dob:");
scanf("%d%d%d",&s[i].d.date,&s[i].d.mon,&s[i].d.year);
printf("\nenter marks:");
scanf("%d%d%d",&s[i].m1,&s[i].m2,&s[i].m3);
s[i].tot=s[i].m1+s[i].m2+s[i].m3;
s[i].avg=s[i].tot/3;
}
printf("\n*** Display the details***\n");
printf("\nrollno\tsname\tdob\ttotal\tavg\n");
for(i=1;i<=n;i++)
{
printf("\%d\t%s\t%d-%d-
%d\t%d\t%f\n",s[i].rollno,s[i].sname,s[i].d.date,s[i].d.mon,s[i].d.year,s[i].tot,s[i].avg);
}
getch();
}
```

b. write short note on Pointers to strings. (May 2014)

Strings are treated like character arrays and therefore, they declared and initialized as follows:

char str[5] = "Good";

The compiler automatically inserts the null character '/0' at the end of the string.

A program using pointers to determine the length of a character string (POINTERS AND CHARACTER STRINGS)

```
#include<stdio.h>
void main()
  {
     char  *name;
     int   length;
     char  *cptr = name;
      name  = "DELHI";
     printf ("%s\n", name);
     while(*cptr != '\0')
     {
        printf("%c is stored at address %u\n", *cptr, cptr);
        cptr++;
```

```
    }
    length = cptr - name;
    printf("\nLength of the string = %d\n", length);
}
```

Output:

```
DELHI
D is stored at address 54
E is stored at address 55
L is stored at address 56
H is stored at address 57
I is stored at address 58
Length of the string = 5
```

Explanation :

A program to count the length of a string is as shown above. The statement

char *cptr = name;

declares **cptr** as a pointer to a character.

Array name by itself is a pointer that means it denotes the address. Hence the above statement **char *cptr = name,** assigns the address of the character array **name** as the initial value to the pointer variable cptr(ie cptr initial value is 54). Since a string is always terminated by the null character, the statement

while(*cptr != '\0')

is true until the end of the string is reached. When the **while** loop is terminated, the pointer **cptr** holds the address of the null character (ie at the end cptr value is 59). Therefore, the statement

length = cptr – name (i.e 59-54 = 5); gives the length of the string **name.**

The output also shows the address location of each character. Note that each character occupies one memory cell (byte).

3. Explain structures in detail. Define a structure data type named Date containing three integer members day, month and year. Design a structure student _record to contain name, date of birth (use Date structure here) and total marks obtained. Develop a c program to read data for 10 students in a class and list them rank-wise. (Jan 2015) (May 2016) (Dec 2016) (Dec 2017) (May 2019) (Dec 2018)

Write a C program that gets and displays the reports of N students with their personal and academic details using structures.

Write a C program to prepare students mark list using structure.

- A Structure is a collection of one or more variables possibly of different data types grouped together under a single name for convenient handling.
- The structure can be declared with the keyword **struct** following the name and opening brace with data elements (members) of different type then closing brace with semicolon, as shown below.

Syntax :-

```
struct  structure_name
{
data-type member1;
data-type member2;
……
……
data-type membern;
}
struct structure_name v1, v2,……vn;
```

- Thus, a single structure may contain **integer elements, floating point elements and character elements, pointers, arrays and other structures** can also be included as elements within a structure.
- Structures help to organize data, particularly in large programs, because they allow a group of related variables to be treated as a single unit.
- A structure is a convenient tool for handling a group of logically related data items.
- For example, it can be used to represent a set of attributes, such as student_name, roll_number, and marks. The concept of a structure is analogous to that of a ‘record’ in may other languages.

- Structure is a type of data structure in which each individual element differs in type. **The elements of a structure are called 'members'**.
- The structure elements contain integer, floating point numbers, character arrays, pointers as their members. Structure act as a tool for handling logically related data items.
- **Fields of structure are called structure elements or members.** Each member may be of different type.

Uses / Applications

- Database Management
- Positioning Cursors
- Receiving ascii and scan codes
- Displaying characters
- Printing on printer
- Mouse Programming
- Graphics Programming
- All Disk Operations

Declaring a Structure:

- Structure is declared using the **keyword 'struct'** followed by a 'tag' and is **enclosed by curly open '{' and close '}' braces**.
- All members of a structure are specified within curly braces { }.

General Form	**struct** structure_name
	{
	data-type member1;
	data-type member2;
	……
	……
	data-type member n;
	};

Description	**structure_name -** refers to the name of the structure. **Member1, member2… member n -** are elements of a structure. **(;) -** Structure is terminated with semicolon.

Example:	struct lib_books
	{
	char book_name[50];
	int pages;
	float price;
	};

Defining a structure

- Defining a structure means **creating a variable to access members of structure.**
- Creating structure variable allows sufficient memory space to hold all the members of structure.

Accessing structure members:

- Members in a structure are accessed using the **Period Operator '.'**.
- Period Operator establishes a link between member and variable name.
- Structure members are processed by using a variable name with period '.' and member name.
- Period Operator is also known as **member operator or dot operator**.

C language does not permit the initialization of individual structure member within the template. The initialization must be done only in the declaration of the actual variables. Note that the compile-time initialization of a structure variable must have the following elements:

1. The keyword struct

2. The structure tag name

3. The name of the variable to be declared

4. The assignment operator =.

5. A set of values for the members of the structure variable, separated by commas and enclosed in braces.

6. A termination with semicolon

Program to store and display student details using structure

```
#include <stdio.h>
struct student {
```

```
    char name[50];
    int roll;
    float marks;
} s;

int main() {
    printf("Enter information:\n");
    printf("Enter name: ");
    fgets(s.name, sizeof(s.name), stdin);

    printf("Enter roll number: ");
    scanf("%d", &s.roll);
    printf("Enter marks: ");
    scanf("%f", &s.marks);

    printf("Displaying Information:\n");
    printf("Name: ");
    printf("%s", s.name);
    printf("Roll number: %d\n", s.roll);
    printf("Marks: %.1f\n", s.marks);

    return 0;
}
```

Output

```
Enter information:
Enter name: Jack
Enter roll number: 23
Enter marks: 34.5
Displaying Information:
Name: Jack
Roll number: 23
Marks: 34.5
```

4. Write a C program using pointers to read two matrices and print their product. (Jan 2015)

Program for matrix multiplication using pointers

```
#include<stdio.h>
int main(){
    int matrix1[20][20],matrix2[20][20],product[20][20];
    int *p1,*p2,*prod,m,n,p,q,d,e,f;
    p1=matrix1;
    p2=matrix2;
    prod=product;
    clrscr();
    printf("\nEnter the size (no of rows and columns) for 1st matrix :\n");
    scanf("%d%d",&m,&n);
    printf("\nEnter the size (no of rows and columns) for 2nd matrix :\n");
    scanf("%d%d",&p,&q);
    if(n==p)
    {
    printf("\n Enter the 1st matrix elements : \n");
    for(d=1;d<=m;++d)
    for(e=1;e<=n;++e)
    scanf("%d",p1+d*20+e);
    printf("\n Enter the 2nd matrix elements : \n\n");
    for(d=1;d<=p;++d)
    for(e=1;e<=q;++e)
    scanf("%d",p2+d*20+e);
    printf("\nThe given 1st Matrix is . . . \n\n");
    for(d=1;d<=m;++d){
    for(e=1;e<=n;++e)
    printf("\t %d", *(p2+d*20+e));
    printf("\n\n");
    }
    printf("\nThe given 2nd Matrix is . . . \n\n");
    for(d=1;d<=p;++d){
    for(e=1;e<=q;++e)
    printf("\t %d", *(p1+d*20+e));
    printf("\n\n");
    }
    for(d=1;d<=m;++d)
```

```
for(e=1;e<=q;++e)
*(prod+d*20+e) = 0;
for(d=1;d<=m;++d)
for(e=1;e<=q;++e)
for(f=1;f<=n;++f)
*(prod+d*20+e) += *(p1+d*20+f) * *(p2+f*20+e);
printf("\n\nThe Product of two matrices using Pointer is . . . \n\n");
for(d=1;d<=m;++d){
for(e=1;e<=q;++e)
printf("\t %d", *(prod+d*20+e));
printf("\n\n");
}
}
else
printf("\nThe Matrix sizes are not compatible for multiplication !!");
getch();
return 0;
}
```

Output:

Enter the size (no of rows and columns) for 1st matrix :4 3

Enter the size (no of rows and columns) for 2nd matrix :3 4

Enter the 1st matrix elements :

2 2 2
2 2 2
2 2 2
2 2 2

Enter the 2nd matrix elements :

2 2 2 2
2 2 2 2
2 2 2 2

The given 1st Matrix is . . .

2 2 2

2 2 2

2 2 2

2 2 2

The given 2nd Matrix is . . .

2 2 2 2

2 2 2 2

2 2 2 2

The Product of two matrices using Pointer is . . .

12 12 12 12

12 12 12 12

12 12 12 12

12 12 12 12

5. Explain in detail about: (Jan 2016) (May 2018)

a) Nested structure
[Refer 2(a)]

b) Array of structures with an example program for each.

- Array is a collection of similar data types. In the same way, we also define array of structures.
- Group of structure variable assigned with **same type stored in an array**. Arrays within structures used to store array of values within a structure.

Syntax	Example
struct structure_name { data_type member1; data_type member2; …… …… data_type member; **}structure_variable[size];**	struct marks { int sub1; int sub2; int sub3; int total; **};** main() { struct marks student[3] = { {45,68,81{, {75,53,69}, {57,36,71}};

The above example declares the student as an array of three elements student[0], student[1], and student[2] and initializes their members as follows

student[0].subject1 = 45;

student[0].subject2 = 65;

…….

…….

student[2].subject3 = 71;

An array of structures is stored inside the memory in the same way as a multi-dimensional array.

The array student actually looks as shown in figure given below:

Figure: The array student inside memory

student[0].subject1	**45**
student[0].subject2	**68**
student[0].subject3	**81**
student[1].subject1	**75**
student[1].subject2	**53**
student[1].subject3	**69**
student[2].subject1	**57**
student[2].subject2	**36**
student[2].subject3	**71**

EXAMPLE PROGRAM FOR ARRAY OF STRUCTURES

```
#include<stdio.h>
{

struct info
{
int id_no;
char name[20];
char address[20];
char combination[3];
int age;
}

struct info std[100];
int i,n;
printf("Enter the number of students");
scanf("%d",&n);
printf(" Enter Id_no,name address combination age\m");
for(i=0;i<n;i++)
scanf(%d%s%s%s",&std[i].id_no,std[i].name,std[i].address,std[i].combination);
scanf("%d", &std[i].age);
```

```
printf("\n Student information");

for(i=0;i<n;i++)
printf("%d%s%s%s \n",,std[i].id_no,std[i].name,std[i].address,std[i].combination);
printf("%d" ,std[i].age);
getch();
}
```

6. Explain in detail about: (Jan 2016)

a) Pointer to an array

Consider the following program,

```
#include<stdio.h>
int main()
{
  int arr[5] = { 1, 2, 3, 4, 5 };
  int *ptr = arr;

  printf("%p\n", ptr);
  return 0;
}
```

In this program, we have a pointer *ptr* that points to the 0^{th} element of the array. Similarly, we can also declare a pointer that can point to whole array instead of only one element of the array. This pointer is useful when talking about multidimensional arrays.

Syntax

data_type (*var_name)[size_of_array];

Example:

```
int (*ptr)[10];
```

Here *ptr* is pointer that can point to an array of 10 integers. Since subscript have higher precedence than indirection, it is necessary to enclose the indirection operator and pointer name inside parentheses.

Here the type of ptr is 'pointer to an array of 10 integers'.

Note : The pointer that points to the 0^{th} element of array and the pointer that points to the whole array are totally different.

C program to understand difference between pointer to an integer and pointer to an array of integers.

```
#include<stdio.h>
int main()
{
   // Pointer to an integer
   int *p;
   // Pointer to an array of 5 integers
   int (*ptr)[5];
   int arr[5];
   // Points to 0th element of the arr.
   p = arr;
   // Points to the whole array arr.
   ptr = &arr;
   printf("p = %p, ptr = %p\n", p, ptr);
   p++;
   ptr++;
   printf("p = %p, ptr = %p\n", p, ptr);
   return 0;
}
```

Output:

p = 0x7fff4f32fd50, ptr = 0x7fff4f32fd50

p = 0x7fff4f32fd54, ptr = 0x7fff4f32fd64

Here, ***p*** is pointer to 0^{th} element of the array *arr*, while ***ptr*** is a pointer that points to the whole array *arr*.

- The base type of *p* is int while base type of *ptr* is 'an array of 5 integers'.
- We know that the pointer arithmetic is performed relative to the base size, so if we write ptr++, then the pointer *ptr* will be shifted forward by 20 bytes.

b) Array of pointers with an example program for each

Array of pointers:

Array of pointers contains collection of address. The address may be the address of variable or array elements.

Example: int *p[3];

Example: Program to find the maximum element in an integer array using pointers.

```
#include <stdio.h>
 int main()
{
  long array[100], *maximum, size, i, location = 1;
  printf("Enter the number of elements in array\n");
  scanf("%ld", &size);
  printf("Enter %ld integers\n", size);

  for ( i = 0 ; i < size ;i++ )
    scanf("%ld", &array[i]);
    maximum  = array;
    *maximum = *array;

  for (i = 1; i < size; i++)
  {
    if (*(array+i) > *maximum)
    {
       *maximum = *(array+i);
       location = i+1;
    }
  }
  printf("Maximum element found at location %ld and it's value is %ld.\n", location,
*maximum);
  return 0;
}
```

Output:

```
Enter the number of elements in array
5
Enter 5 integers
56
23
46
97
52
Maximum element found at location 4 and it's value is 97
```

7. A) What do you mean by pointer? Mention its uses. (4) (May 2016)

A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address.

The general form of a pointer variable declaration is −

type *var-name;

Here, type is the pointer's base type; it must be a valid C data type and var-name is the name of the pointer variable.

The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer.

The unary or monadic operator & gives the "address of a variable"".

The indirection or dereference operator * gives the "contents of an object pointed to by a pointer".

Uses of pointers:

- Pointers are used for file handling.
- Pointers are used to allocate memory dynamically.
- To pass arguments by reference
- For accessing array elements
- To return multiple values
- To implement data structures
- To do system level programming where memory addresses are useful

B) Write a program to find the sum and average of 10 numbers using pointers (7) (May 2016)

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int n,*p,sum=0,i;
    float avg;
    printf("Number of numbers:: ");
    scanf("%d",&n);
    p=(int *) malloc(sizeof(int));
    if(p==NULL)
    {
        printf("\nMemory Allocation unsuccessful.\n");
        exit(0);
```

```
    }
    for(i=0;i<n;i++)
    {
        printf("\nEnter Number %d :: ",i+1);
        scanf("%d",p+i);
    }
  for(i=0;i<n;i++)
  {
   sum=sum+*(p+i);
  }
  printf("\nThe Sum of %d Numbers is %d \n",n,sum);
    avg=(float)sum/n;
    printf("\nThe Average of %d Numbers is %0.2f \n",n,avg);
    return 0;
}
```

Output:

```
Number of numbers :: 6
Enter Number 1 :: 1
Enter Number 2 :: 2
Enter Number 3 :: 3
Enter Number 4 :: 4
Enter Number 5 :: 5
Enter Number 6 :: 6
The Sum of 6 Numbers is 21
The Average of 6 Numbers is 3.50
Process returned 0
```

8. Write a C program using pointer to read a 2 x 2 matrix and find its inverse (Dec 2016)

C program to find inverse of a matrix

```
#include<stdio.h>
#include<math.h>
float determinant(float [][25], float);
void cofactor(float [][25], float);
void transpose(float [][25], float [][25], float);
int main()
{
```

```
  float a[25][25], k, d;
  int i, j;
  printf("Enter the order of the Matrix : ");
  scanf("%f", &k);
  printf("Enter the elements of %.0fX%.0f Matrix : \n", k, k);
  for (i = 0;i < k; i++)
    {
     for (j = 0;j < k; j++)
       {
        scanf("%f", &a[i][j]);
        }
    }
  d = determinant(a, k);
  if (d == 0)
   printf("\nInverse of Entered Matrix is not possible\n");
  else
   cofactor(a, k);
}

/*For calculating Determinant of the Matrix */
float determinant(float a[25][25], float k)
{
  float s = 1, det = 0, b[25][25];
  int i, j, m, n, c;
  if (k == 1)
    {
     return (a[0][0]);
    }
  else
    {
     det = 0;
     for (c = 0; c < k; c++)
       {
        m = 0;
        n = 0;
        for (i = 0;i < k; i++)
          {
            for (j = 0 ;j < k; j++)
              {
                b[i][j] = 0;
```

```
            if (i != 0 && j != c)
             {
              b[m][n] = a[i][j];
              if (n < (k - 2))
               n++;
              else
               {
                n = 0;
                m++;
               }
             }
           }
        det = det + s * (a[0][c] * determinant(b, k - 1));
        s = -1 * s;
        }
   }

   return (det);
}

void cofactor(float num[25][25], float f)
{
 float b[25][25], fac[25][25];
 int p, q, m, n, i, j;
 for (q = 0;q < f; q++)
 {
  for (p = 0;p < f; p++)
   {
    m = 0;
    n = 0;
    for (i = 0;i < f; i++)
    {
     for (j = 0;j < f; j++)
      {
       if (i != q && j != p)
       {
        b[m][n] = num[i][j];
        if (n < (f - 2))
         n++;
```

```
            else
             {
               n = 0;
               m++;
               }
             }
          }
        }
        fac[q][p] = pow(-1, q + p) * determinant(b, f - 1);
      }
    }
    transpose(num, fac, f);
}
/Finding transpose of matrix/
void transpose(float num[25][25], float fac[25][25], float r)
{
  int i, j;
  float b[25][25], inverse[25][25], d;

  for (i = 0;i < r; i++)
    {
     for (j = 0;j < r; j++)
       {
         b[i][j] = fac[j][i];
        }
    }
  d = determinant(num, r);
  for (i = 0;i < r; i++)
    {
     for (j = 0;j < r; j++)
       {
        inverse[i][j] = b[i][j] / d;
        }
    }
   printf("\n\n\nThe inverse of matrix is : \n");

   for (i = 0;i < r; i++)
    {
     for (j = 0;j < r; j++)
       {
```

```
            printf("\t%f", inverse[i][j]);
           }
       printf("\n");
        }
}
```

Output:

Enter the order of the Square Matrix : 3

Enter the elements of 3X3 Matrix : 3 5 2 1 5 8 3 9 2

The inverse of matrix is :

```
 0.704545   -0.090909   -0.340909
-0.250000   -0.000000    0.250000
 0.068180    0.136364   -0.113636
```

9. Write a C program to find the addition and subtraction of two 3 x 3 matrix. (May 2017)

C program to find the addition and subtraction of two 3 x 3 matrix

```
#include<stdio.h>
int main()
{
   int n, m, c, d, first[10][10], second[10][10], sum[10][10], diff[10][10];
   printf("\nEnter the number of rows and columns of the first matrix \n\n");
   scanf("%d%d", &m, &n);
   printf("\nEnter the %d elements of the first matrix \n\n", m*n);
   for(c = 0; c < m; c++)   // to iterate the rows
      for(d = 0; d < n; d++)   // to iterate the columns
         scanf("%d", &first[c][d]);
   printf("\nEnter the %d elements of the second matrix \n\n", m*n);
   for(c = 0; c < m; c++)   // to iterate the rows
      for(d = 0; d < n; d++)   // to iterate the columns
         scanf("%d", &second[c][d]);
   /*
      printing the first matrix
   */
   printf("\n\nThe first matrix is: \n\n");
   for(c = 0; c < m; c++)   // to iterate the rows
```

```
{
   for(d = 0; d < n; d++)   // to iterate the columns
   {
      printf("%d\t", first[c][d]);
   }
printf("\n");
}

/*
   printing the second matrix
*/
printf("\n\nThe second matrix is: \n\n");
for(c = 0; c < m; c++)   // to iterate the rows
{
   for(d = 0; d < n; d++)   // to iterate the columns
   {
      printf("%d\t", second[c][d]);
   }
printf("\n");
}
/*
   finding the SUM of the two matrices
   and storing in another matrix sum of the same size
*/
for(c = 0; c < m; c++)
   for(d = 0; d < n; d++)
      sum[c][d] = first[c][d] + second[c][d];
// printing the elements of the sum matrix
printf("\n\nThe sum of the two entered matrices is: \n\n");
for(c = 0; c < m; c++)
{
   for(d = 0; d < n; d++)
   {
      printf("%d\t", sum[c][d]);
   }
   printf("\n");
}
/*
   finding the DIFFERENCE of the two matrices
   and storing in another matrix difference of the same size
```

```
    */
    for(c = 0; c < m; c++)
        for(d = 0; d < n; d++)
            diff[c][d] = first[c][d] - second[c][d];
    // printing the elements of the diff matrix
    printf("\n\nThe difference(subtraction) of the two entered matrices is: \n\n");
    for(c = 0; c < m; c++)
    {
        for(d = 0; d < n; d++)
        {
            printf("%d\t", diff[c][d]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

Enter the number of rows and columns of the first matrix

3

3

Enter the 9 elements of the first matrix

1 4 3 6 7 4 0 8 4

Enter the 9 elements of the second matrix

2 4 5 9 6 7 0 3 4

The first matrix is :

1 4 3

6 7 4

0 8 4

The second matrix is :

2 4 5

9 6 7

0 3 4

The sum of the two entered matrices is :

3 8 8

15 13 11

0 11 8

The difference(subtraction) of the two entered matrices is:

-1 0 -2

-3 1 -3

0 5 0

10. What do you mean by nested structure? Write a C program to pass a copy of entire structure to a function. (Dec 2017)

- Nesting of structures are nothing but **structure within another structure** (i.e., a structure can contain one or more structures as its members).
- A structure may be defined and/or declared inside another structure.

Syntax	**Example**
struct structure_name1 { <data-type> member 1; <data-type> member 2; - <data-type> member n; **struct structur_ name2** { <data-type> member 1; <data-type> member 2; - <data-type> member n;	struct stud_Res { int rno; char nm[50]; char std[10]; struct stud_subj { char subjnm[30]; int marks; }subj; }result;

}inner_struct_var; }outer_struct_var;	

Description:

- In above example, the structure stud_Res consists of stud_subj which itself is a structure with two members.
- Structure stud_Res is called as '**outer structure**' while stud_subj is called as '**inner structure.**'
- The members which are inside the inner structure can be accessed as follow :

  result.subj.subjnm

  result.subj.marks

EXAMPLE PROGRAM – PASSING STRUCTURE TO FUNCTION IN C BY VALUE:

```
#include <stdio.h>
#include <string.h>

struct student
{
        int id;
        char name[20];
        float percentage;
};

void func(struct student record);

int main()
{
        struct student record;

        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        func(record);
        return 0;
}
```

```
void func(struct student record)
{
        printf(" Id is: %d \n", record.id);
        printf(" Name is: %s \n", record.name);
        printf(" Percentage is: %f \n", record.percentage);
}
```

Output:

```
Id is: 1
Name is: Raju
Percentage is: 86.500000
```

EXAMPLE PROGRAM – PASSING STRUCTURE TO FUNCTION IN C BY ADDRESS:

```
#include <stdio.h>
#include <string.h>

struct student
{
        int id;
        char name[20];
        float percentage;
};

void func(struct student *record);

int main()
{
        struct student record;

        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        func(&record);
        return 0;
}
```

```
void func(struct student *record)
{
        printf(" Id is: %d \n", record->id);
        printf(" Name is: %s \n", record->name);
        printf(" Percentage is: %f \n", record->percentage);
}
```

Output:

```
Id is: 1
Name is: Raju
Percentage is: 86.500000
```

11. Write a program using pointers to determine length of a character string. (May 2018)

Program to determine length of a string using pointers

```
#include<stdio.h>
#include<conio.h>
int string_ln(char*);
void main() {
 char str[20];
 int length;
 clrscr();
 printf("\nEnter any string : ");
 gets(str);
  length = string_ln(str);
   printf("The length of the given string %s is : %d", str, length);
   getch();
}
 int string_ln(char*p) /* p=&str[0] */
{
   int count = 0;
   while (*p != '\0') {
      count++;
      p++;
   }
   return count;
```

```
}
```

Output:

```
Enter the String : pritesh
Length of the given string pritesh is : 7
```

Explanation:

1. **gets()** is used to accept string with spaces.
2. we are **passing accepted string to the function**.
3. Inside function we have **stored this string in pointer**. (i.e base of the string is stored inside pointer variable).
4. Inside while loop we are going to count single letter and incrementing pointer further till **we get null character.**

12. List out the features of pointers. Write a C program to multiply two numbers using pointers. (Dec 2018)

Features of Pointers:

1. Pointers save memory space.
2. Execution time with pointers is faster because data are manipulated with the address, that is ,direct access to memory location.
3. Memory is accessed efficiently with the pointers. The pointer assigns and releases the memory as well. Hence it can be said the Memory of pointers is dynamically allocated.
4. Pointers are used with data structures. They are useful for representing two-dimensional and multi-dimensional arrays.
5. An array, of any type can be accessed with the help of pointers, without considering its subscript range.
6. Pointers are used for file handling.
7. Pointers are used to allocate memory dynamically.
8. In C++, a pointer declared to a base class could access the object of a derived class. However, a pointer to a derived class cannot access the object of a base class.

Program to multiply two numbers using pointers

A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address.

The general form of a pointer variable declaration is −

type *var-name;

Here, type is the pointer's base type; it must be a valid C data type and var-name is the name of the pointer variable.The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. The unary or monadic operator & gives the "address of a variable'". The indirection or dereference operator * gives the "contents of an object pointed to by a pointer".

13. Explain how structures are different from arrays. Justify using suitable example. (May 2019)

Array	**Structure**
Array refers to a collection consisting of elements of homogenous data type.	Structure refers to a collection consisting of elements of heterogenous data type.
Array uses subscripts or "[ ]" (square bracket) for element access.	Structure uses "." (Dot operator) for element access.
Array is pointer as it points to the first element of the collection.	Structure is not a pointer
Instantiation of Array objects is not possible.	Instantiation of Structure objects is possible.
Array size is fixed and is basically the number of elements multiplied by the size of an element.	Structure size is not fixed as each element of Structure can be of different type and size.
Bit field is not possible in an array.	Bit field is possible in a structure.
Array declaration is done simply using [] and not any keyword.	Structure declaration is done with the help of "struct" keyword.
Array is a primitive datatype	Structure is a user-defined datatype.
Array traversal and searching is easy and fast.	Structure traversal and searching is complex and slow.

data_type array_name[size];	struct struct_name{
	data_type1 ele1;
	data_type2 ele2;
	};

Array elements are stored in continuous memory locations.	Structure elements may or may not be stored in a continuous memory location.
Array elements are accessed by their index number using subscripts.	Structure elements are accessed by their names using dot operator.

Example program for array

A pointer is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address.

The general form of a pointer variable declaration is −

type *var-name;

Here, type is the pointer's base type; it must be a valid C data type and var-name is the name of the pointer variable. The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. The unary or monadic operator & gives the "address of a variable". The indirection or dereference operator * gives the "contents of an object pointed to by a pointer".

14. What is the difference between union and structure? Explain with an example. (Dec 2019)

Structure	**Union**
• Every member has its own memory space. • Keyword struct is used • Any member can be accessed at any time without the loss of data • It can handle all the members or a few as required at a time • It may be initialized with all its members • More memory space is used	• All the members use the same memory space to store the values • Keyword union is used • Different interpretations for the same memory location is possible • It can handle only one member at time, even all the members use the same space • Only one of its members may be initialized • Conservation of memory is possible

Example program for structure

```
#include <stdio.h>
struct student {
   char name[60];
   int roll_no;
   float marks;
} sdt;
    int main() {
   printf("Enter the following information:\n");
   printf("Enter student name: ");
   fgets(sdt.name, sizeof(sdt.name), stdin);
   printf("Enter student roll number: ");
   scanf("%d", & sdt. roll_no);
   printf("Enter students marks: ");
   scanf("%f", & sdt.marks);
   printf("The information you have entered is: \n");
   printf("Student name: ");
   printf("%s", sdt.name);
   printf("Student roll number: %d\n", sdt. roll_no);
   printf("Student marks: %.1f\n", sdt.marks);
   return 0;
}
```

Output:

Enter the following information:

Enter student name: James

Enter student roll number: 21

Enter student marks: 67

The information you have entered is:

Student name: John

Student roll number: 21

Student marks: 67.0

In the above program, a structure called student is created. This structure has three data members: 1) name (string), 2) roll_no (integer), and 3) marks (float). After this, a structure variable sdt is created to store student information and display it on the computer screen.

Example program for Union

```
#include <stdio.h>

union item
{
   int x;
   float y;
   char ch;
};

int main( )
{
   union item it;
   it.x = 12;
   it.y = 20.2;
   it.ch = 'a';

   printf("%d\n", it.x);
   printf("%f\n", it.y);
   printf("%c\n", it.ch);

   return 0;
}
```

Output:

1101109601

20.199892

In the above program, you can see that the values of x and y gets corrupted. Only variable ch prints the expected result. It is because, in union, the memory location is shared among all member data types. Therefore, the only data member whose value is currently stored, will occupy memory space. The value of the variable ch was stored at last, so the value of the rest of the variables is lost.

15. How to pass a pointer to a function? Explain with an example. (Dec 2019)

- Pointer to a function contains the address of function in memory. Pointers are passed to a function as arguments, helps to use it as argument in another function. The general form for declaring pointer to a function is

Syntax :-

returntype (*fptr)();

where fptr is a pointer to a function. fptr returns the value of type return type.

POINTERS AS FUNCTION PARAMETERS

```
void exchange (int *, int *);     /* prototype */

main()

{

   int  x, y;

   x = 100;

   y = 200;

   printf("Before exchange  : x = %d   y = %d\n\n", x, y);

   exchange(&x,&y);          /* call */

   printf("After exchange   : x = %d   y = %d\n\n", x, y);

}
```

```
exchange (int *a, int *b)
{
   int t;
   t = *a;    /* Assign the value at address a to t */
   *a = *b;   /* put b into a */
   *b = t;    /* put t into b */
}
```

Output:

```
Before exchange  : x = 100   y = 200
After exchange   : x = 200   y = 100
```

Note:-

Pointer to a function is simply "**a pointer whose value is the address of the function name"**.